



Lab 04

COMP1322 Programming II

Student Name: Qi Liu

Student ID: 36557854

Student Email: ql1e24@soton.ac.uk

Date: March 9, 2026

1: Understanding Object-Oriented Programming in Java

1.1 Key Definitions

Dynamic Method Dispatch means that Java can decide which method to run while the program is running. Besides, a parent class reference can store a child class object. When a method is called, Java will check the real object type, instead of the reference type, then it will run the correct method from that object.

Example code for dynamic method dispatch:

```
Person a = new person();  
a.sleep();
```

But if we override a person with child, then my programme will run child to sleep instead of person to sleep.

Method overriding is when a child class makes its own version of a method that is already in the parent class. But they have to make sure that the method names, parameters, and return types are the same, otherwise it won't work perfectly.

Example code for method overriding:

```
public class Dispatch {  
    public static void main(String[] args) { Human human = new Human ();  
        sayHello(human);  
        Human woman = new Woman ();  
        sayHello (woman);  
        Human man = new Man ();  
        sayHello(man);  
        public static void sayHello (Human human){  
            human. sayHello();  
        }  
    }  
}
```

Polymorphism means a parent reference can represent different child objects in our programme.

For example:

```
Driver d;  
  
d = new Max();  
  
d.overtake();  
  
d = new George();  
  
d.overtake();
```

Here, both Max and George are the different child objects of parent Driver reference.

1.2 Why Abstract Classes and Interfaces are Useful

Because an abstract class is like a template (blueprint) for other classes, but it cannot create any kind of objects. It can provide some basic rules or shared code to other classes, but if we want to create a new class with the template of the abstract class, we have to fulfill the code by ourselves.

Interface is useful because it tells what kind of methods must exist if we want call it. And it also doesn't provide any detail codes of method.

2. Dynamic Method Dispatch

2.1 Code

```
1  abstract class Employee { 5个用法 3个继承者  
2      String name; 5个用法  
3      double salary; 2个用法  
4  
5      Employee(String name, double salary) { 3个用法  
6          this.name = name;  
7          this.salary = salary;  
8      }  
9  
10     abstract void work(); 1个用法 3个实现  
11  
12     void showDetails() { 1个用法  
13         System.out.println("Name: " + name + " | Salary: $" + salary); // common method for all employees to show their details  
14     }  
15 }  
16  
17 class Developer extends Employee { 1个用法  
18     Developer(String n, double s) { 1个用法  
19         super(n, s); // calls the constructor of Employee  
20     }  
21  
22     void work() { 1个用法  
23         System.out.println(name + " - Writes code :");  
24     }  
25 }  
26  
27 class Manager extends Employee { 1个用法  
28     Manager(String n, double s) { 1个用法  
29         super(n, s); // calls the constructor of Employee  
30     }  
31  
32     void work() { 1个用法  
33         System.out.println(name + " - Manages team");  
34     }  
35 }  
36  
37 class Tester extends Employee { 1个用法  
38     Tester(String n, double s) { 1个用法  
39         super(n, s);  
40     }  
41     void work() { 1个用法  
42         System.out.println(name + " - Tests software :");  
43     }  
44 }
```

```

43 }
44
45 public class Task2_DynamicDispatch {
46     public static void main(String[] args) {
47         Employee[] e = {
48             new Developer( n: "Skim", s: 67700),
49             new Manager( n: "Max", s: 90000),
50             new Tester( n: "Lando", s: 65000)
51         };
52
53         for (Employee emp : e) {
54             emp.showDetails(); emp.work(); // call work() behaves differently based on Developer, Manager, Tester
55         }
56     }
57 }

```

2.2 Output

```

Name: Skim | Salary: $67700.0
Skim - Writes code :)
Name: Max | Salary: $90000.0
Max - Manages team
Name: Lando | Salary: $65000.0
Lando - Tests software :)

```

2.3 Explanation of Dynamic Method Dispatch

Dynamic Method Dispatch means that the right overridden method would be chosen based on the actual type of the object when we run the code, instead of the type of the reference. For example, here,

Skim's reference is Employee, but the object is Developer, so work() prints "Code writes :)"

Max's reference is Employee, but the object is Manager, so work() prints "Manages team"

Lando's reference is Employee, but the object is Tester. When you call work(), it prints "Tests software :)"

3. Concrete Classes

3.1 Code

```

interface Vehicle { 5个用法 3个实现
    void start(); // every class implementing Vehicle MUST define start() 1个用法
    void stop(); // same for stop() 1个用法 3个实现
}

class Car implements Vehicle { 1个用法

    @Override 1个用法
    public void start() { System.out.println("Car is starting..."); }

    @Override 1个用法
    public void stop() { System.out.println("Car is stopping..."); }
}

class Hyundai implements Vehicle { 1个用法
    @Override 1个用法
    public void start() {
        System.out.println("Your Hyundai Elantra 1.8cc Sports is starting...");
    }

    @Override 1个用法
    public void stop() {
        System.out.println("Your Hyundai Elantra 1.8 cc Sports is stopping...");
    }
}

class Truck implements Vehicle { 1个用法
    @Override 1个用法
    public void start() {
        System.out.println("Truck is starting...");
    }

    @Override 1个用法
    public void stop() {
        System.out.println("Truck is stopping...");
    }
}

```

```

public class Task3_Interfaces {
    public static void main(String[] args) {
        Vehicle[] vehicles = {
            new Car(),    // vehicle reference pointing to a Car object
            new Hyundai(), // pointing to my Hyundai object
            new Truck(),   // pointing to a Truck object
        };

        // loop through all vehicles and call their start and stop methods
        for (Vehicle v : vehicles) {
            v.start();
            v.stop();
            System.out.println("---");
        }
    }
}

```

3.2 Output

```

Car is starting...
Car is stopping...
---
Your Hyundai Elantra 1.8 cc Sports is starting...
Your Hyundai Elantra 1.8 cc Sports is stopping...
---
Truck is starting...
Truck is stopping...
---

```

3.3 Why Can't We Instantiate an Interface?

Because it doesn't define how methods are implemented, so there is nothing to instantiate.

4. Overview of UML

4.1 Use Case Diagram

1. Identify the actors involved in this scenario.

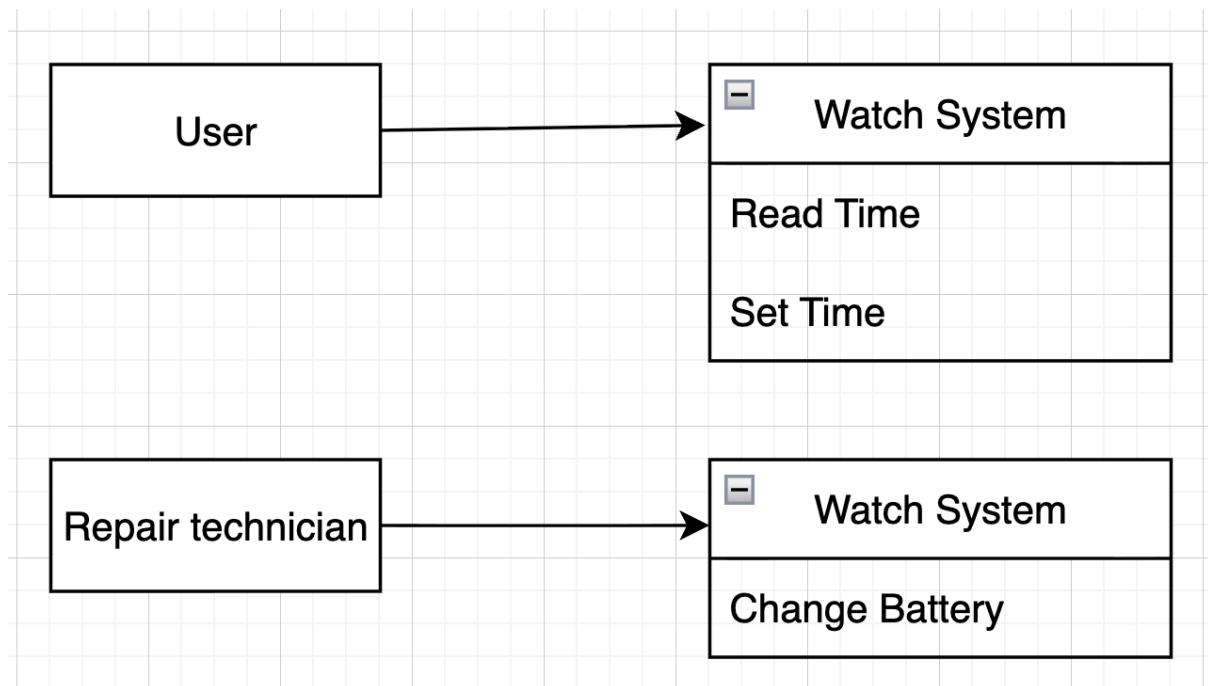
User: using our watch

Repair technician: changing the battery to restore functionality

2. Determine the use cases based on the described interactions.

Read time, set time and change the battery to restore functionality.

3. Draw a Use Case Diagram representing the interactions between the identified actors and the watch system.



4.2 Scenario: Class Diagram

1. Identify the key classes in this system.

Watch, Push Buttons, an LCD Displays, Battery and a Time Module.

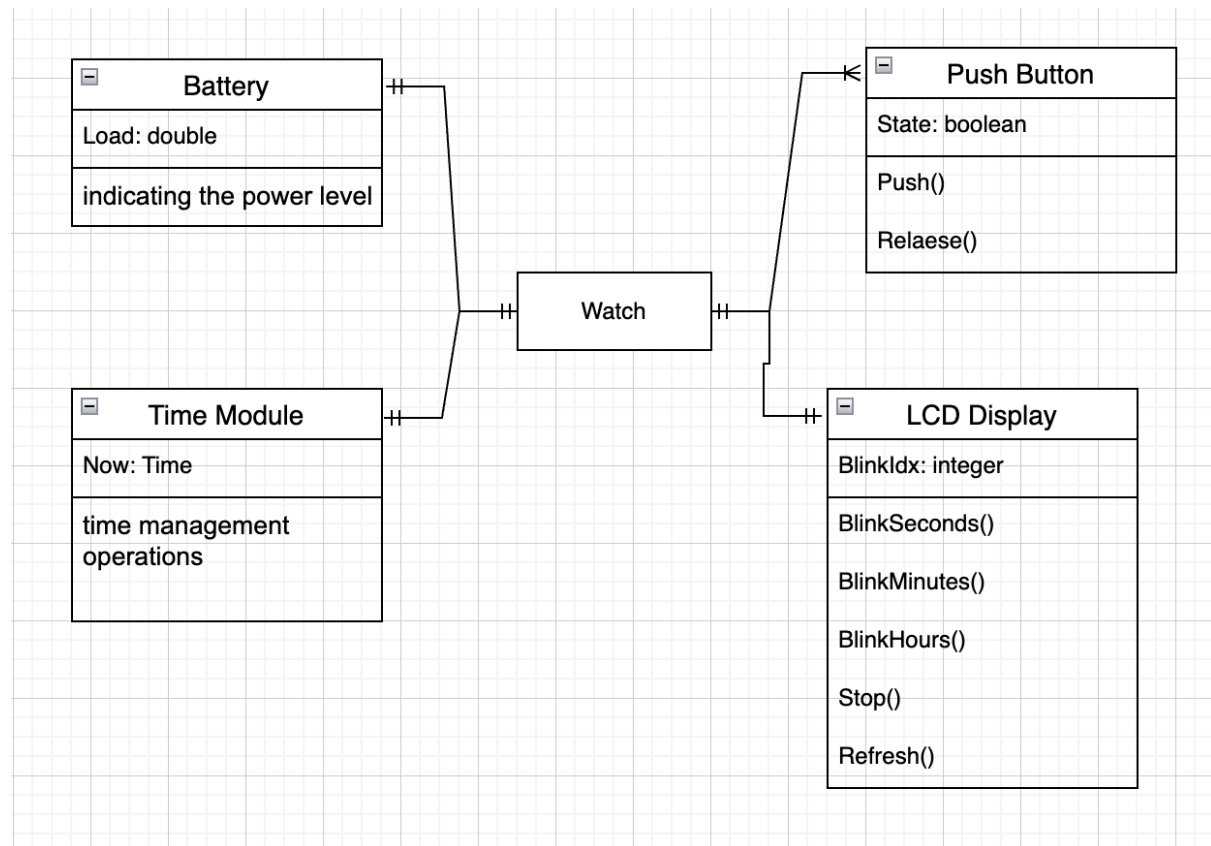
2. Determine the attributes and operations associated with each class.

Class	Attributes	Operations
Watch	Scheduling every components inside itself	uses PushButton, LCDDisplay, TimeModule, Battery
PushButton	state: boolean	push() release()
LCD Display	blinkIdx: integer	blinkSeconds() blinkMinutes() blinkHours() stop() refresh()
Battery	load: double	indicating the power level
Time Module	now: Time	time management operations

3. Establish associations and multiplicities between the classes.

- 1 Watch has 2 Push Button to set up state.
- 1 Watch has 1 LCD Display to display the information.
- 1 Watch has at least 1 Battery to use.
- 1 Watch has 1 Time Module to manage the time.

4. Draw a UML Class Diagram representing the structure of this watch system



4.3 Scenario: Sequence Diagram

1. Identify the actor involved in the interaction.

Watch User.

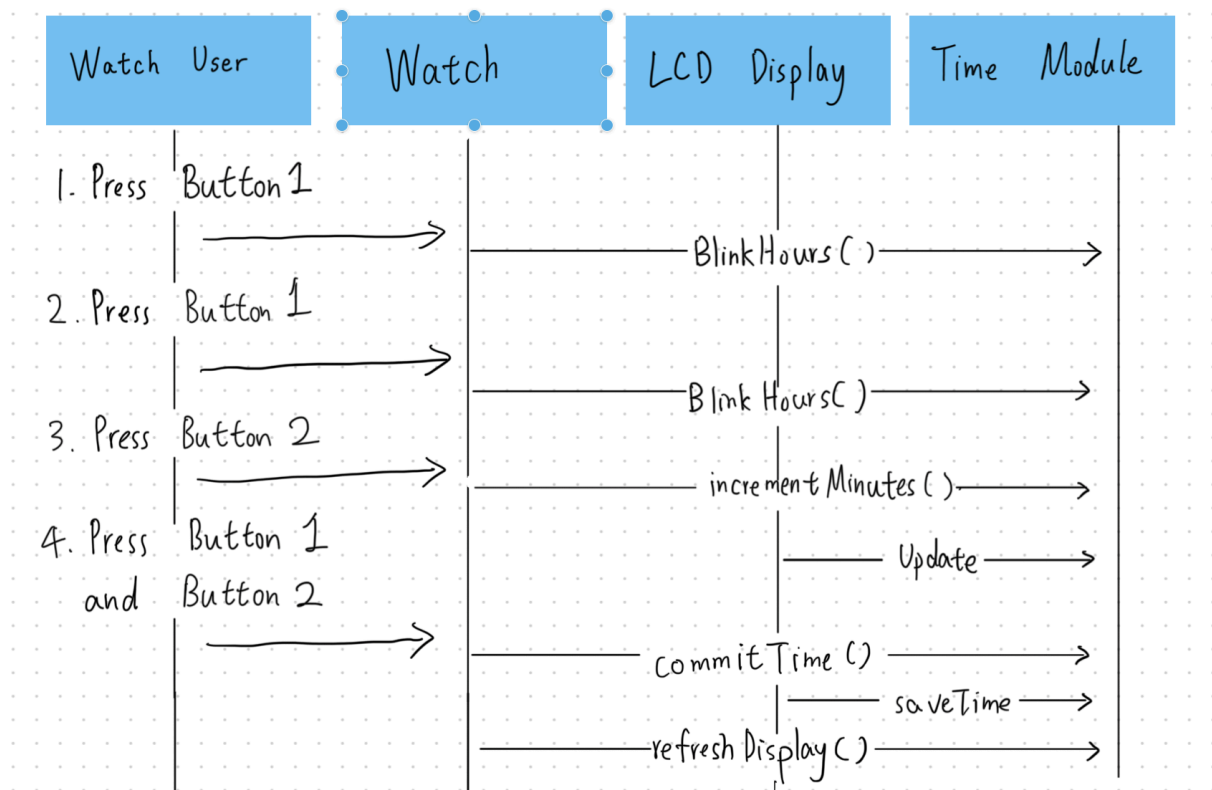
2. Determine the key objects that participate in the message exchanges.

2 Push Buttons, LCD Displays, battery and Time Module.

3. List the sequence of messages and responses exchanged between the watch system components.

1. User presses Button 1 → Watch sends blinkHours() to LCD Display
2. User presses Button 1 again → Watch sends blinkMinutes() to LCD Display
3. User presses Button 2 → Watch sends setMinutes(+1) to Time Module
4. User presses Button 1 + Button 2 together → Watch sends commitTime() to Time Module, then stop() and refresh() to LCD Display

4. Draw a UML Sequence Diagram that visually represents the step-by- step interaction between the Watch User, Watch, LCD Display, and Time module.



2. Programming task

Code:

```

1  import java.util.Scanner;
2
3  interface Withdrawable { 5个用法 3个实现
4      void withdraw(double amount); // only accounts that can withdraw will implement this 1个用法 3个实现
5  }
6
7  interface Depositable { 4个用法 2个实现
8      void deposit(double amount); // deposit 1个用法 2个实现
9  }
10
11  abstract class BankAccount { 5个用法 3个继承者
12      String accountNumber; 2个用法
13      String accountHolder; 2个用法
14      double balance; 20个用法
15
16      BankAccount(String n, String h, double b) { 3个用法
17          accountNumber=n;
18          accountHolder=h;
19          balance=b;
20      }
21
22      double getBalance() { 0个用法
23          return balance; // common method to get balance for all accounts
24      }
25
26      void displayAccountInfo() { 1个用法
27          System.out.println("Account Holder: " + accountHolder);
28          System.out.println("Account Number: " + accountNumber);
29          System.out.println("Current Balance: $" + balance);
30      }
31
32      abstract void calculateInterest(); 1个用法 3个实现
33  }
34
35  class SavingsAccount extends BankAccount implements Withdrawable, Depositable { // Savings account can withdraw and deposit 1个用法
36      SavingsAccount(String n, String h, double b) { 1个用法
37          super(n, h, b); // call the constructor of BankAccount
38      }
39
40      public void withdraw(double a) { 1个用法
41          if (balance - a < 500) { // check if balance would drop below minimum of $500
42              System.out.println("Cannot withdraw! Minimum balance is $500.");
43          }
44          else {
45              balance -= a;

```

```

44         balance -= a;
45         System.out.println("Withdrawal successful! New Balance: $" + balance);
46     }
47 }
48 public void deposit(double a) { 1个用法
49     balance += a;
50     System.out.println("Deposit successful! New Balance: $" + balance);
51 }
52
53 void calculateInterest() { 1个用法
54     balance *= 1.02; // 2% interest for savings account
55     System.out.println("Interest added! New Balance: $" + balance);
56 }
57 }
58
59 class CurrentAccount extends BankAccount implements Withdrawable, Depositable { 1个用法
60     CurrentAccount(String n, String h, double b) { 1个用法
61         super(n, h, b); // call the constructor of BankAccount
62     }
63
64     public void withdraw(double a) { 1个用法
65         balance -= a;
66         System.out.println("Withdrawal successful! New Balance: $" + balance);
67     }
68
69     public void deposit(double a) { 1个用法
70         balance += a;
71         System.out.println("Deposit successful! New Balance: $" + balance);
72     }
73
74     void calculateInterest() { 1个用法
75         System.out.println("No interest for Current Account. :)");
76     }
77 }
78
79 class CreditAccount extends BankAccount implements Withdrawable { 1个用法
80     double creditLimit = 5000; 1个用法
81     CreditAccount(String n, String h, double b) { 1个用法
82         super(n, h, b);
83     }

```

```

85  public void withdraw(double a) { 1个用法
86      if (balance - a < -creditLimit) { // if withdrawal exceeds credit limit
87          System.out.println("Credit limit exceeded! :(");
88      }
89      else {
90          balance -= a; // allow balance to go negative up to credit limit
91          System.out.println("Withdrawal successful! New Balance: $" + balance);
92      }
93  }
94  void calculateInterest() { // if balance is negative, charge 5% interest 1个用法
95      if (balance < 0) {
96          balance *= 1.05;
97          System.out.println("Interest charged! New Balance: $" + balance);
98      }
99  }
100 }
101
102 public class ATMSystem {
103     public static void main(String[] args) {
104         Scanner sc = new Scanner(System.in);
105         BankAccount[] accounts = {
106             new SavingsAccount(n: "Saving-001", h: "Skim", b: 1500),
107             new CurrentAccount(n: "Current-002", h: "Max", b: 3000),
108             new CreditAccount(n: "Credit-003", h: "Lando", b: 0)
109         };
110         int choice;
111
112         do {
113             System.out.println("\nSelect an account:\n1. Skim\n2. Max\n3. Lando\n0. Exit");
114             choice = sc.nextInt();
115             if (choice == 0) { // if user chooses to exit
116                 System.out.println("ATM System Closed. Thanks for using.");
117                 break;
118             }
119             BankAccount acc = accounts[choice - 1];
120             acc.displayAccountInfo();
121             if (acc instanceof Withdrawable) { // if the account can withdraw, ask for amount to withdraw
122                 System.out.print("Enter amount to withdraw: ");
123                 ((Withdrawable) acc).withdraw(sc.nextDouble());
124             }
125             if (acc instanceof Depositable) { // if the account can deposit, ask for amount to deposit
126                 System.out.print("Enter amount to deposit: ");

```

```

102 public class ATMSystem {
103     public static void main(String[] args) {
104         Scanner sc = new Scanner(System.in);
105         BankAccount[] accounts = {
106             new SavingsAccount( n: "Saving-001", h: "Skim", b: 1500),
107             new CurrentAccount( n: "Current-002", h: "Max", b: 3000),
108             new CreditAccount( n: "Credit-003", h: "Lando", b: 0)
109         };
110         int choice;
111
112         do {
113             System.out.println("\nSelect an account:\n1. Skim\n2. Max\n3. Lando\n0. Exit");
114             choice = sc.nextInt();
115             if (choice == 0) { // if user chooses to exit
116                 System.out.println("ATM System Closed. Thanks for using.");
117                 break;
118             }
119             BankAccount acc = accounts[choice - 1];
120             acc.displayAccountInfo();
121             if (acc instanceof Withdrawable) { // if the account can withdraw, ask for amount to withdraw
122                 System.out.print("Enter amount to withdraw: ");
123                 ((Withdrawable) acc).withdraw(sc.nextDouble());
124             }
125             if (acc instanceof Depositatable) { // if the account can deposit, ask for amount to deposit
126                 System.out.print("Enter amount to deposit: ");
127                 ((Depositatable) acc).deposit(sc.nextDouble());
128             }
129             acc.calculateInterest();
130         } while (choice != 0);
131         sc.close();
132     }
133 }

```

Output:

Select an account:

- 1. Skim
- 2. Max
- 3. Lando
- 0. Exit

1

Account Holder: Skim

Account Number: Saving-001

Current Balance: \$1500.0

Enter amount to withdraw: 500

Withdrawal successful! New Balance: \$1000.0

Enter amount to deposit: 677

Deposit successful! New Balance: \$1677.0

Interest added! New Balance: \$1710.54